

Лабораторна робота №2 Багатопоковість. Асинхроність. IEnumerable. LINQ.

Виконав студент групи 302-ТК

Рогочий Михайло

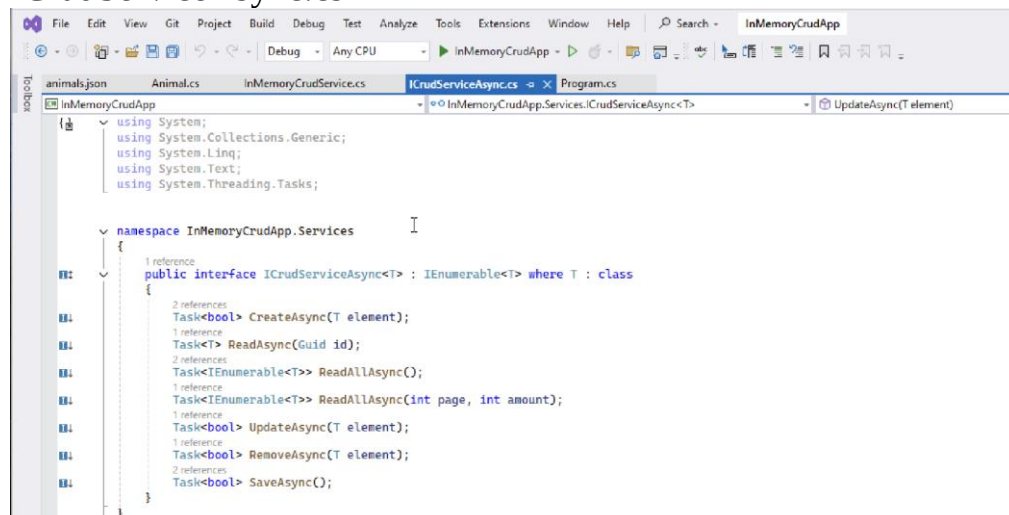
Передумова:

Завдання

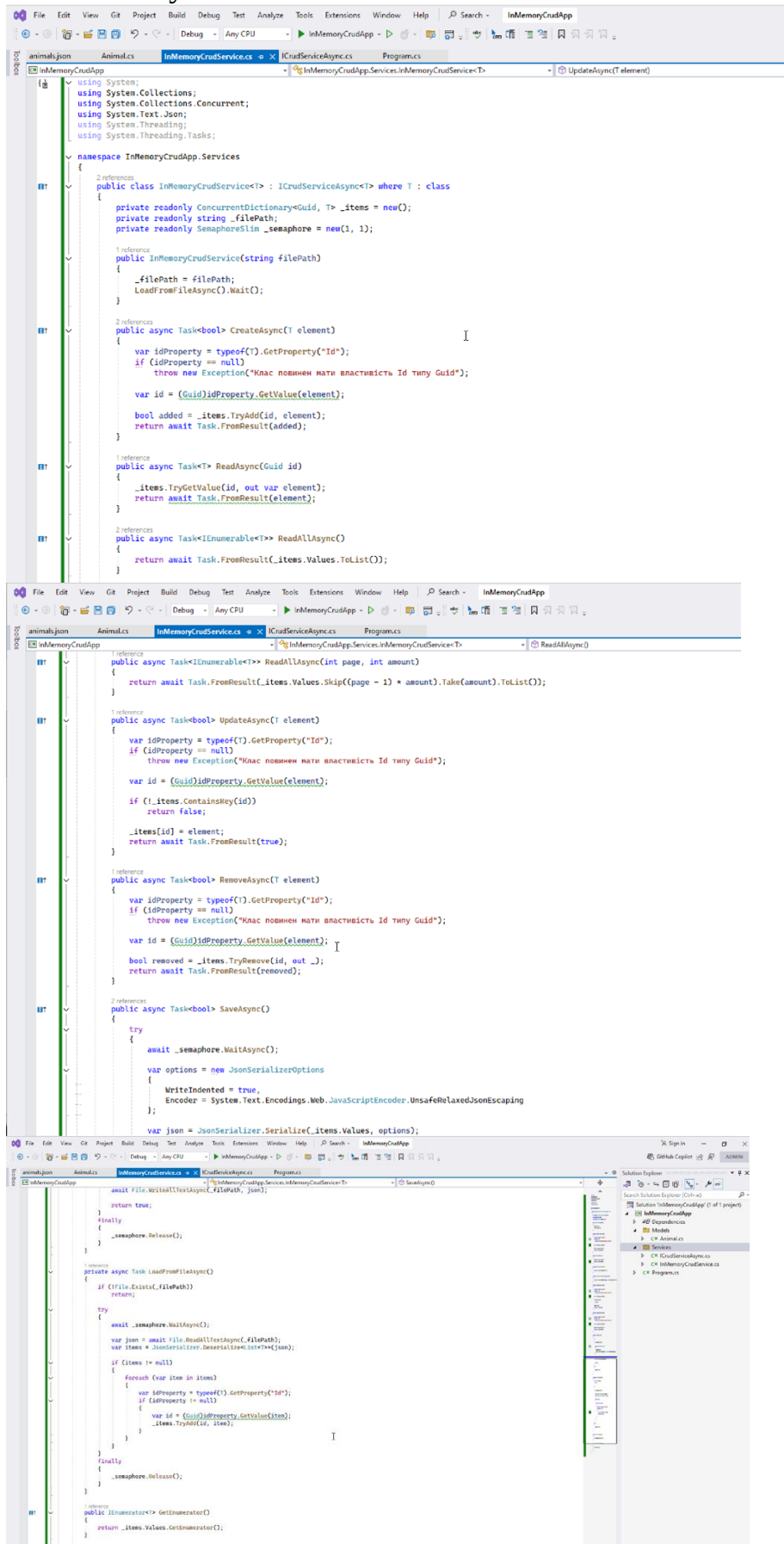
1.Реалізувати асинхронну версію дженерік CRUD сервісу, який буде зберігати дані у одній із вбудованих колекцій .NET, буде багатопотоково-безпечною (thread safe), зберігатиме асинхронно колекцію у серіалізованому вигляді у файлі за шляхом FilePath, матиме вбудовану підтримку пагінації та реалізовуватиме наступний інтерфейс та інтерфейс IEnumerable:

```
public interface ICrudServiceAsync<T> : IEnumerable<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int
amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}
```

Для цього створюємо папку Services та створюємо у ній два файли:
ICrudServiceAsync.cs



Ta InMemoryCrudService.cs



```

0 references
IEnumerator IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}
}
}

```

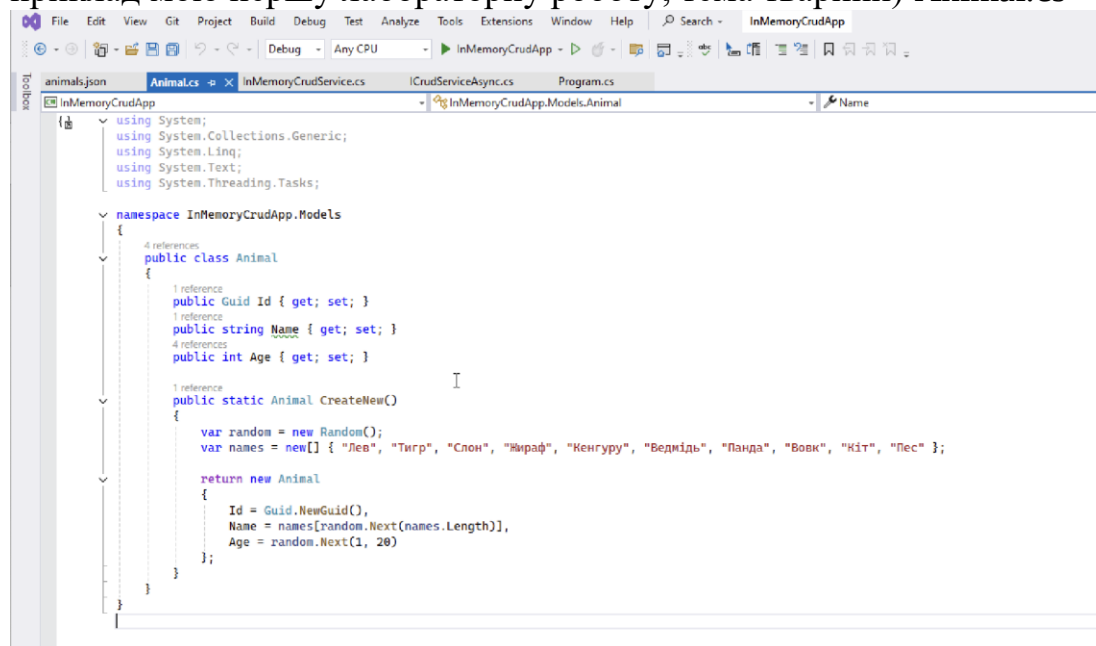
2. Створіть статичні методи у класах, які створюватимуть нові об'єкти цього класу із згенерованими даними:

```

public class Bus
{
    public static Bus CreateNew()
    {
        throw new NotImplementedException();
    }
}

```

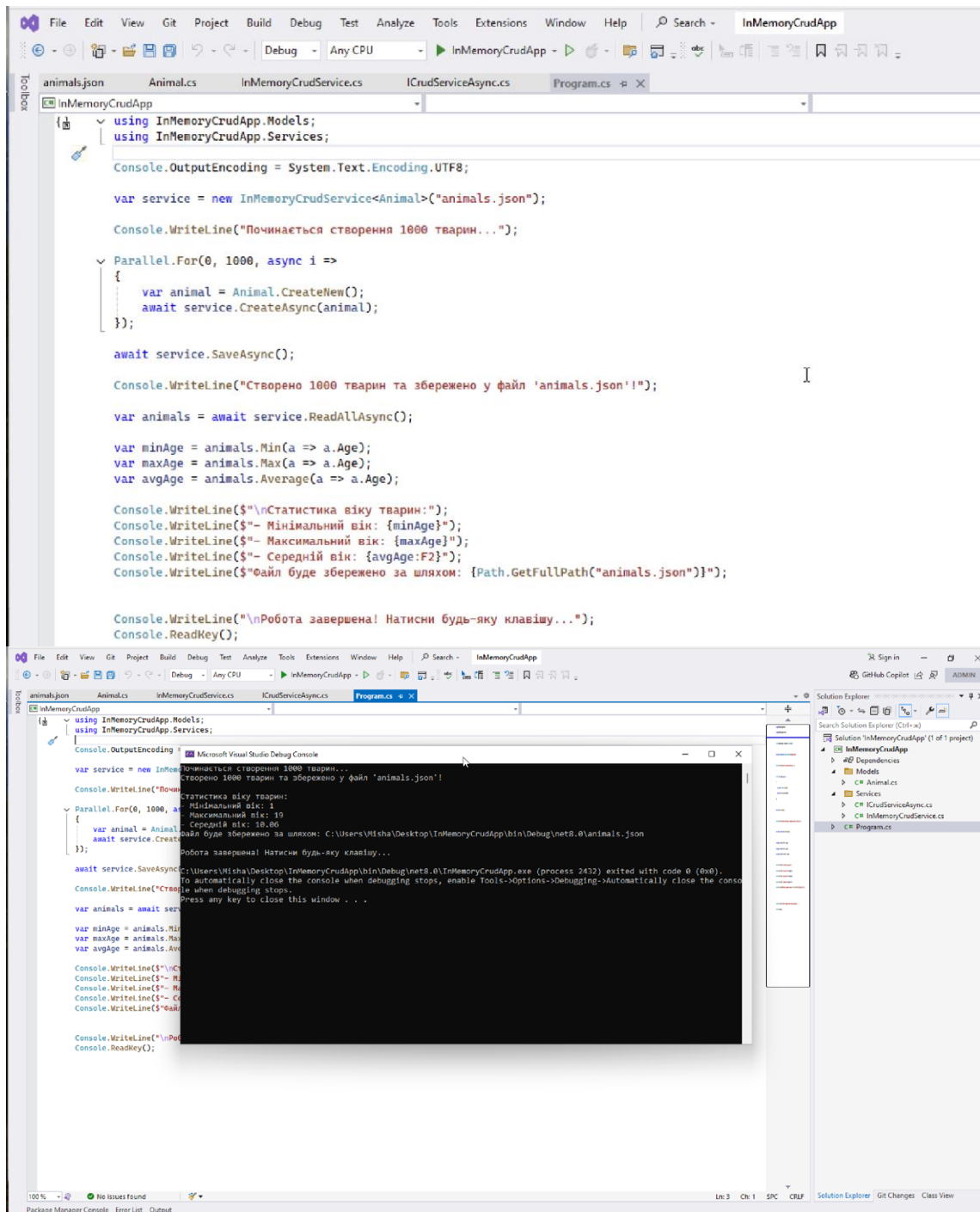
Тут створюємо папку Models та створюємо там файл з об'єктами (Я взяв за приклад мою першу лабораторну роботу, тема тварини) Animal.cs



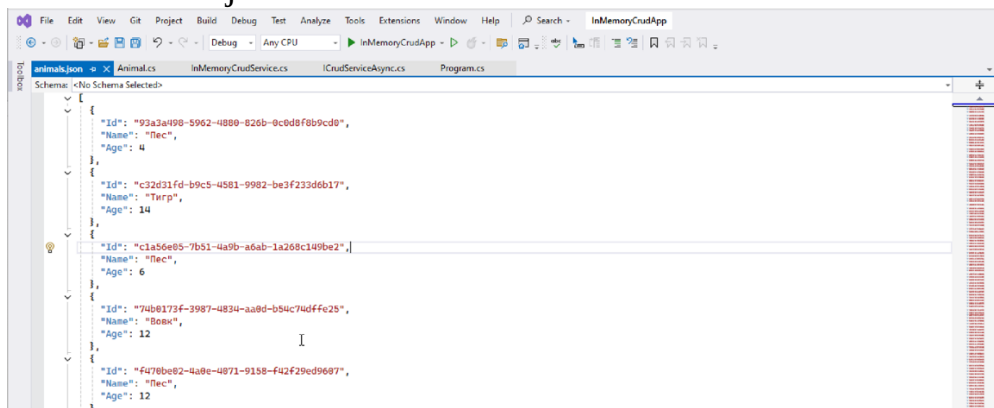
3. Модифікуйте консольний застосунок, щоб він паралельно створив від тисячі об'єктів використовуючи CRUD service, створені у об'єктах метод CreateNew та клас Parallel. Для створених об'єктів знайдіть мінімальні, максимальні та середні значення для цифрових значень, отримані результати виведіть у консоль. Згенеровану колекцію збережіть у файл.

Для цього пишемо код у файл Program.cs.

Коротко що він робить: створює 1000 тварин з різними комбінаціями назв віку та айді і записує все у файл animals.json



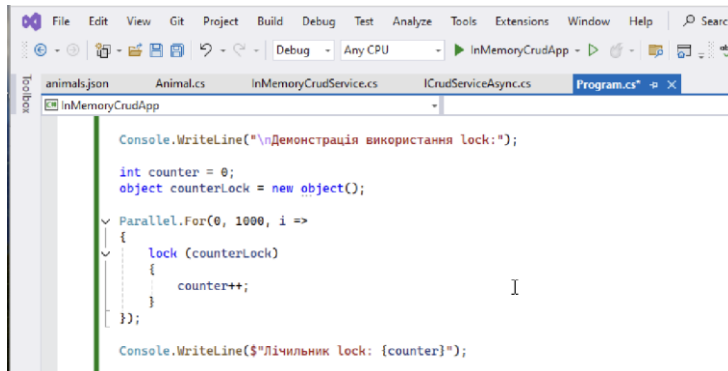
Файл animals.json:



4. Додайте приклади використання примітивів синхронізації як Lock, Semaphore, AutoResetEvent та інші.

Працюємо у тому ж Program.cs

Lock:



```
File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search
Debug Any CPU InMemoryCrudApp
animals.json Animals InMemoryCrudService.cs ICrudServiceAsync.cs Program.cs
InMemoryCrudApp

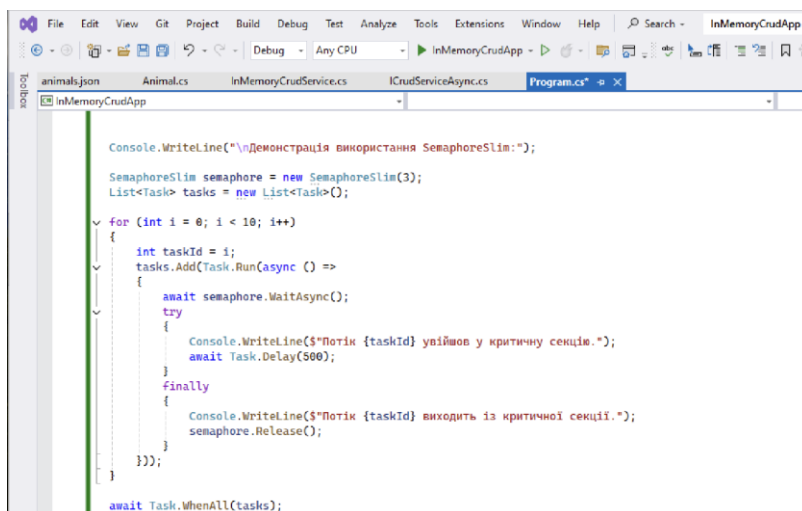
Console.WriteLine("\nДемонстрація використання lock:");

int counter = 0;
object counterLock = new object();

Parallel.For(0, 1000, i =>
{
    lock (counterLock)
    {
        counter++;
    }
});

Console.WriteLine($"Лічильник lock: {counter}");
```

Semaphore:



```
File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search InMemoryCrudApp
Debug Any CPU InMemoryCrudApp
animals.json Animals InMemoryCrudService.cs ICrudServiceAsync.cs Program.cs
InMemoryCrudApp

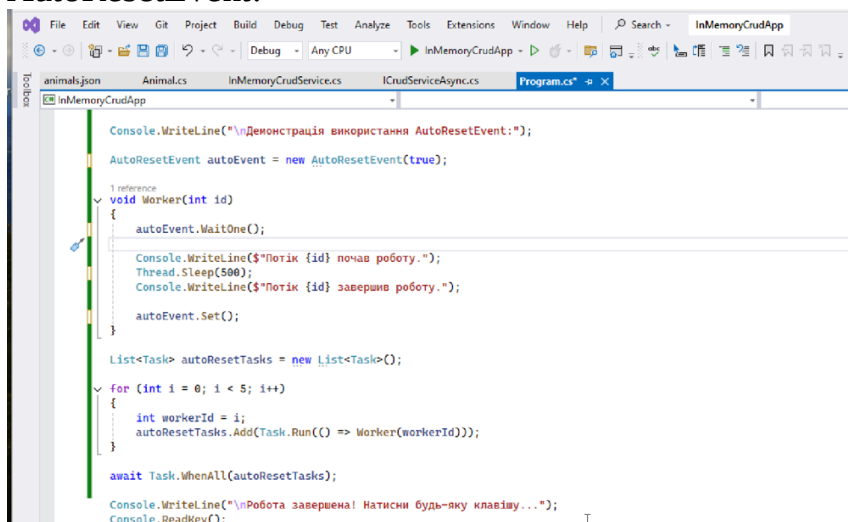
Console.WriteLine("\nДемонстрація використання SemaphoreSlim:");

SemaphoreSlim semaphore = new SemaphoreSlim(3);
List<Task> tasks = new List<Task>();

for (int i = 0; i < 10; i++)
{
    int taskId = i;
    tasks.Add(Task.Run(async () =>
    {
        await semaphore.WaitAsync();
        try
        {
            Console.WriteLine($"Потік {taskId} увійшов у критичну секцію.");
            await Task.Delay(500);
        }
        finally
        {
            Console.WriteLine($"Потік {taskId} виходить із критичної секції.");
            semaphore.Release();
        }
    }));
}

await Task.WhenAll(tasks);
```

AutoResetEvent:



```
File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search InMemoryCrudApp
Debug Any CPU InMemoryCrudApp
animals.json Animals InMemoryCrudService.cs ICrudServiceAsync.cs Program.cs
InMemoryCrudApp

Console.WriteLine("\nДемонстрація використання AutoResetEvent:");

AutoResetEvent autoEvent = new AutoResetEvent(true);

1 reference
void Worker(int id)
{
    autoEvent.WaitOne();

    Console.WriteLine($"Потік {id} почав роботу.");
    Thread.Sleep(500);
    Console.WriteLine($"Потік {id} завершив роботу.");

    autoEvent.Set();
}

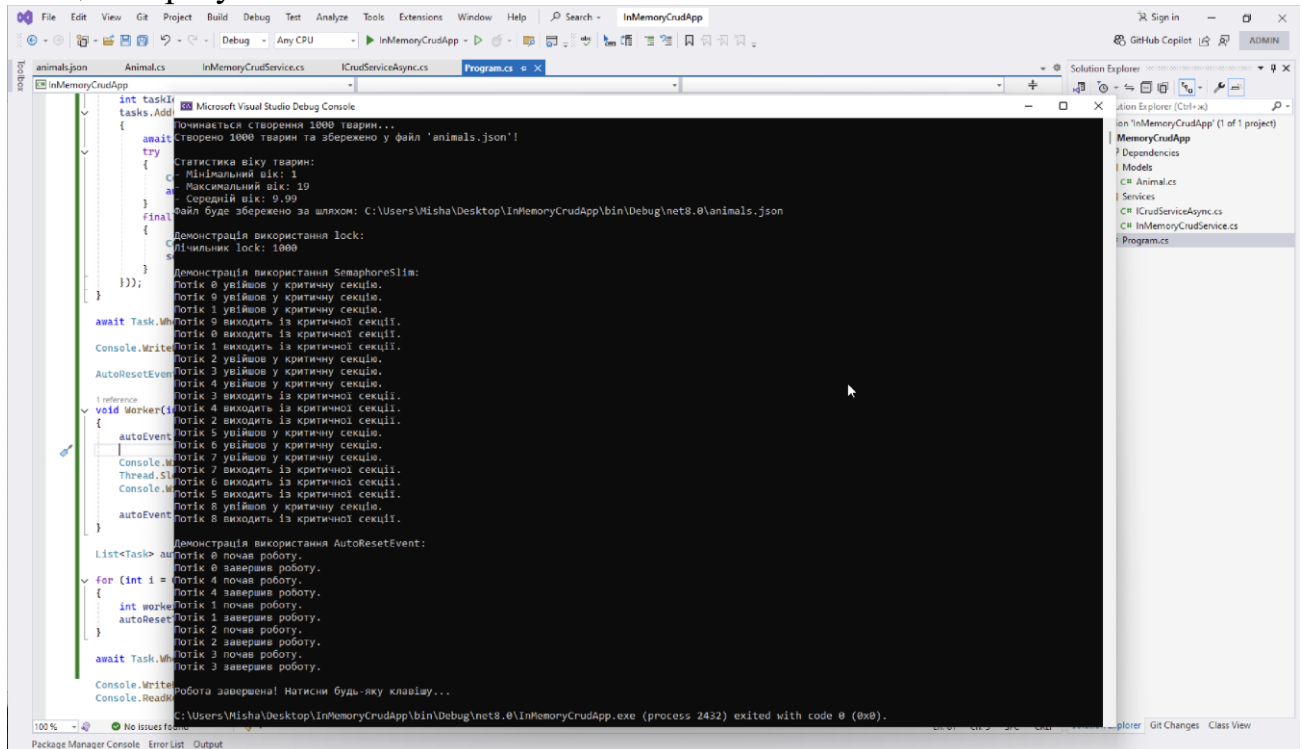
List<Task> autoResetTasks = new List<Task>();

for (int i = 0; i < 5; i++)
{
    int workerId = i;
    autoResetTasks.Add(Task.Run(() => Worker(workerId)));
}

await Task.WhenAll(autoResetTasks);

Console.WriteLine("\nРобота завершена! Натисни будь-яку клавішу...");
Console.ReadKey();
```

Кінцевий результат:

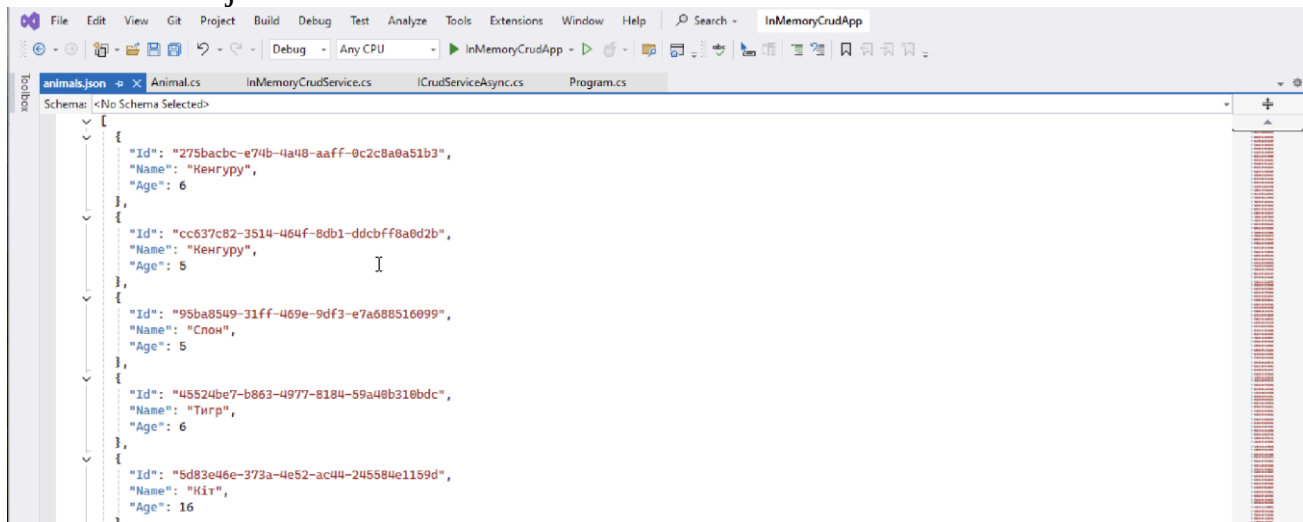


```
int tasks.Add(
{
    await Task.Run(() =>
    {
        // Створюємо 1000 тварин та зберігаємо у файл 'animals.json'!
        try
        {
            // Статистика віку тварин:
            Console.WriteLine("Мінімальний вік: 1");
            Console.WriteLine("Максимальний вік: 19");
            Console.WriteLine("Середній вік: 9.99");
        }
        finally
        {
            // Файл буде збережено за шляхом: C:\Users\Misha\Desktop\InMemoryCrudApp\bin\Debug\net8.0\animals.json
        }
    });

    // Демонстрація використання lock:
    // Починаємо lock: 1000
    // Демонстрація використання SemaphoreSlim:
    // Потік 0 увійшов у критичну секцію.
    // Потік 0 вийшов з критичної секції.
    // Потік 1 увійшов у критичну секцію.
    // Потік 1 вийшов з критичної секції.
    // Console.WriteLine("Потік 0 вийшов з критичної секції.");
    // Console.WriteLine("Потік 1 вийшов з критичної секції.");
    // Потік 2 увійшов у критичну секцію.
    // AutoResetEvent
    // Потік 3 увійшов у критичну секцію.
    // Потік 4 увійшов у критичну секцію.
    // Потік 3 вийшов з критичної секції.
    // Потік 4 вийшов з критичної секції.
    // Потік 2 вийшов з критичної секції.
    // Потік 5 увійшов у критичну секцію.
    // Потік 5 вийшов з критичної секції.
    // Потік 7 увійшов у критичну секцію.
    // Console.WriteLine("Потік 7 вийшов з критичної секції.");
    // Console.WriteLine("Потік 6 вийшов з критичної секції.");
    // Thread.Sleep(1000);
    // Потік 6 вийшов з критичної секції.
    // autoEvent
    // Потік 8 вийшов з критичної секції.

    // Демонстрація використання AutoResetEvent:
    // Потік 0 почав роботу.
    // Потік 0 завершив роботу.
    // for (int i = 0; i < 4; i++)
    // {
    //     // Потік 4 почав роботу.
    //     // Потік 4 завершив роботу.
    //     int workerId = i;
    //     autoResetEvent
    //     // Потік 1 почав роботу.
    //     // Потік 1 завершив роботу.
    //     // Потік 2 почав роботу.
    //     // Потік 2 завершив роботу.
    //     // Потік 3 почав роботу.
    //     // Потік 3 завершив роботу.
    // }
    Console.WriteLine("Робота завершена! Натисни будь-яку клавішу...");
    Console.ReadLine();
}
```

Файл animals.json

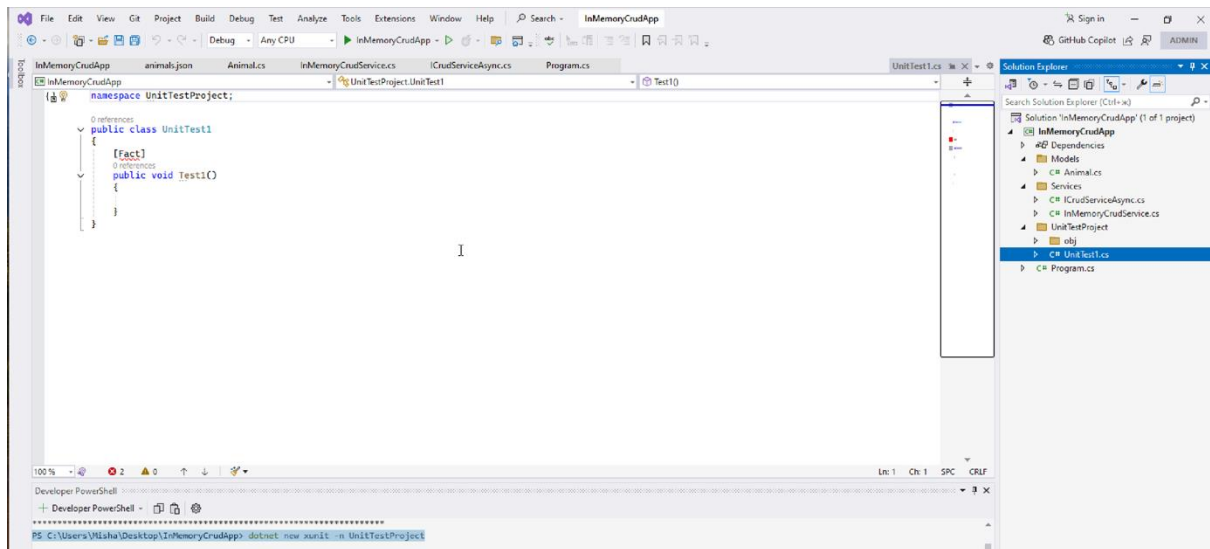


```
[
  {
    "Id": "275bacbc-e74b-4a48-aaff-0c2c8a0a51b3",
    "Name": "Кенгуру",
    "Age": 6
  },
  {
    "Id": "cc637c82-3514-464f-8db1-ddcbff8a0d2b",
    "Name": "Кенгуру",
    "Age": 5
  },
  {
    "Id": "95ba8549-31ff-469e-9df3-e7a688516099",
    "Name": "Слон",
    "Age": 5
  },
  {
    "Id": "45524be7-b863-4977-8184-59a48b318bdc",
    "Name": "Тигр",
    "Age": 6
  },
  {
    "Id": "5d83e46e-373a-4e52-ac44-245584e1159d",
    "Name": "Кіт",
    "Age": 16
  }
]
```

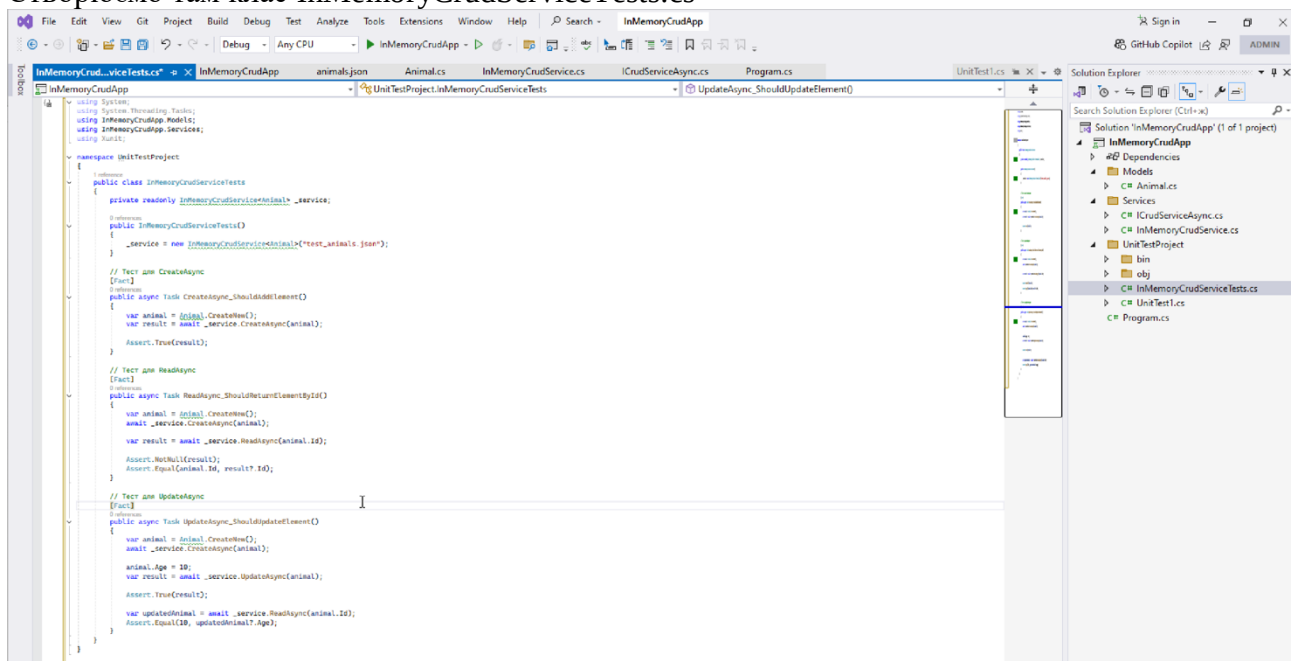
Додаткове завдання

Створіть модульні (Unit) тести для створеною вами реалізації `ICrudServiceAsync<T>`.

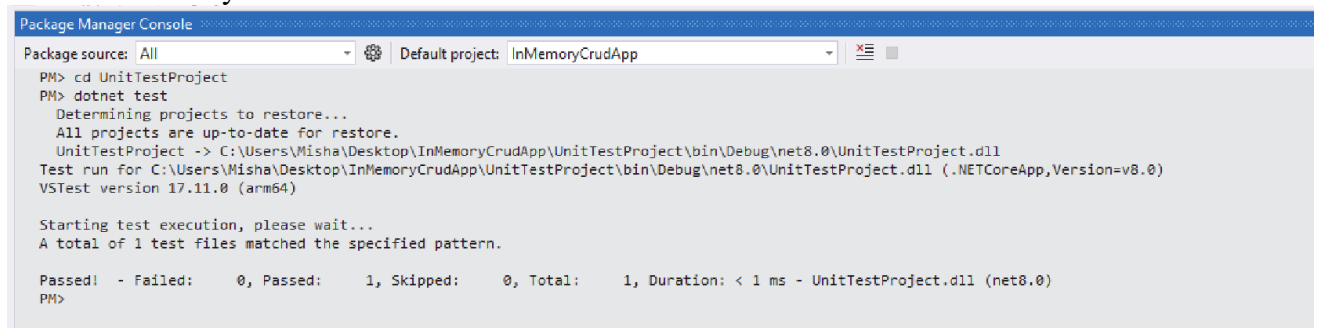
Для цього створюємо новий проект у нашому файлі через термінал
Додаємо потрібні пакети
Та додаємо йому посилання на наш основний проект, щоб тестовий міг використовувати класи та методи з основного проекту

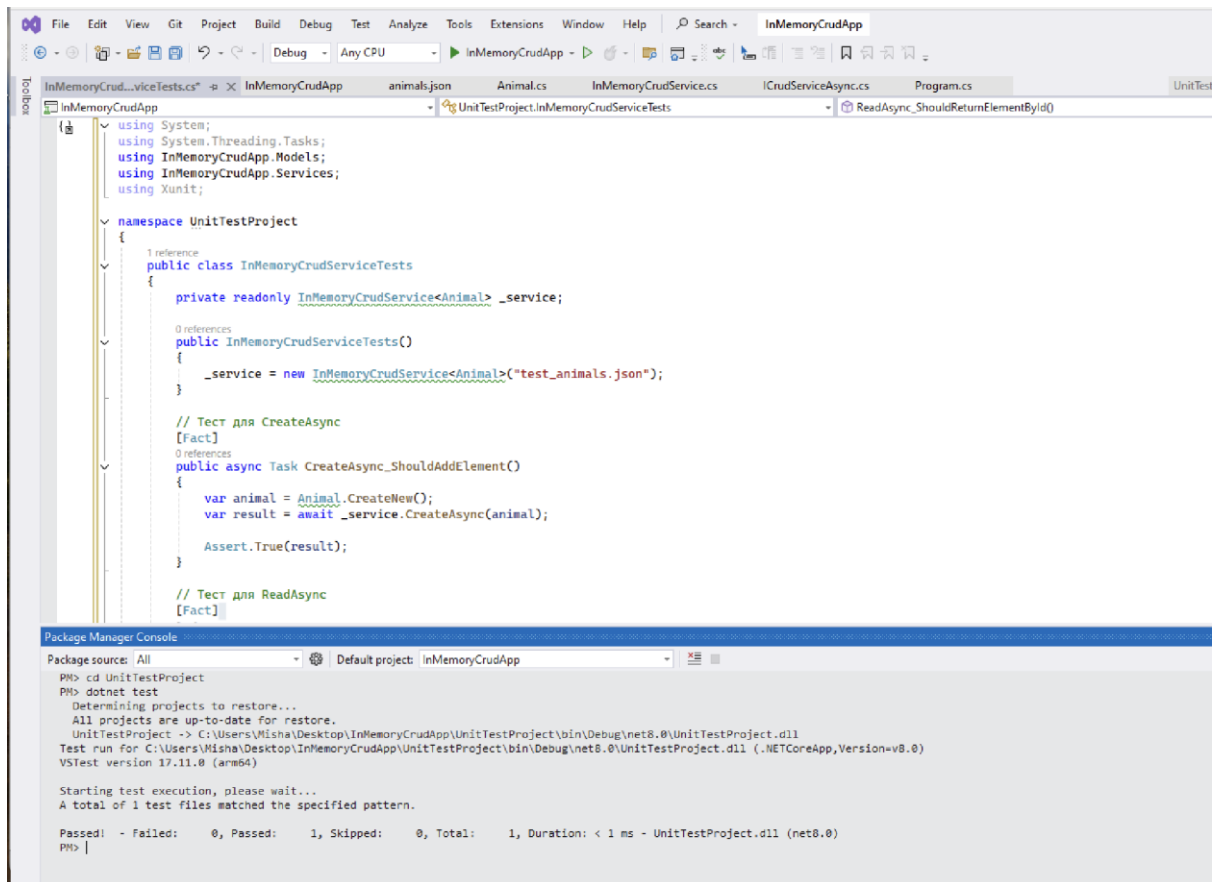


Створюємо там клас InMemoryCrudServiceTests.cs

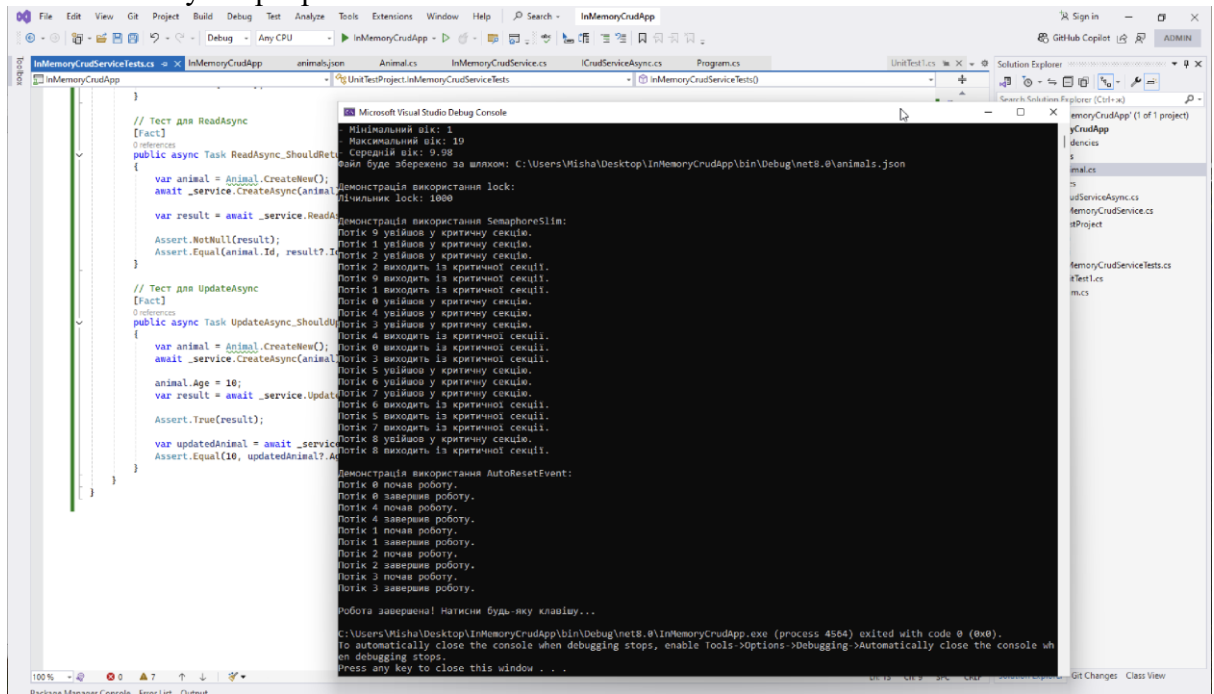


Виконання тесту:





Останній запуск програми



Все працює коректно